Integration Testing Report

StreetRace Manager

Assignment 2

March 24, 2026

Contents

1 System Overview **2**

2 2.1 Integration Call Graph **2**

3 2.2 Integration Test Design **2**

3.1 Test Cases 2

4 Execution Summary **5**

5 Observations **5**

1 System Overview

StreetRace Manager is a modular command-line application.

Required modules implemented:

1. Registration Module
2. Crew Management Module
3. Inventory Module
4. Race Management Module
5. Results Module
6. Mission Planning Module

Additional modules implemented:

1. Maintenance Module
2. Reputation Module

2 2.1 Integration Call Graph

The integration call graph for StreetRace Manager is captured below:

This call graph models key method-level interactions across integration modules:

- system facade wiring in `StreetRaceSystem`
- race entry dependency chain across registration and inventory modules
- result recording propagation to race closure, reputation, and cash inventory
- mission-planning dependency checks over registration and inventory state
- maintenance dependency checks for mechanic, parts, tools, and repair state update

3 2.2 Integration Test Design

3.1 Test Cases

1. **Register driver and enter race**
   Modules: Registration, Crew Management, Inventory, Race Management.
   Expected result: registered driver is accepted and appears in participant list.
   Actual result: PASS.
   Errors/issues found: None.
   Why needed: validates primary cross-module race-entry workflow.

2. **Reject unregistered race participant**
   Modules: Registration, Race Management.
   Expected result: entry is blocked with validation error.
   Actual result: PASS.
   Errors/issues found: None.
   Why needed: enforces registration-before-racing rule.

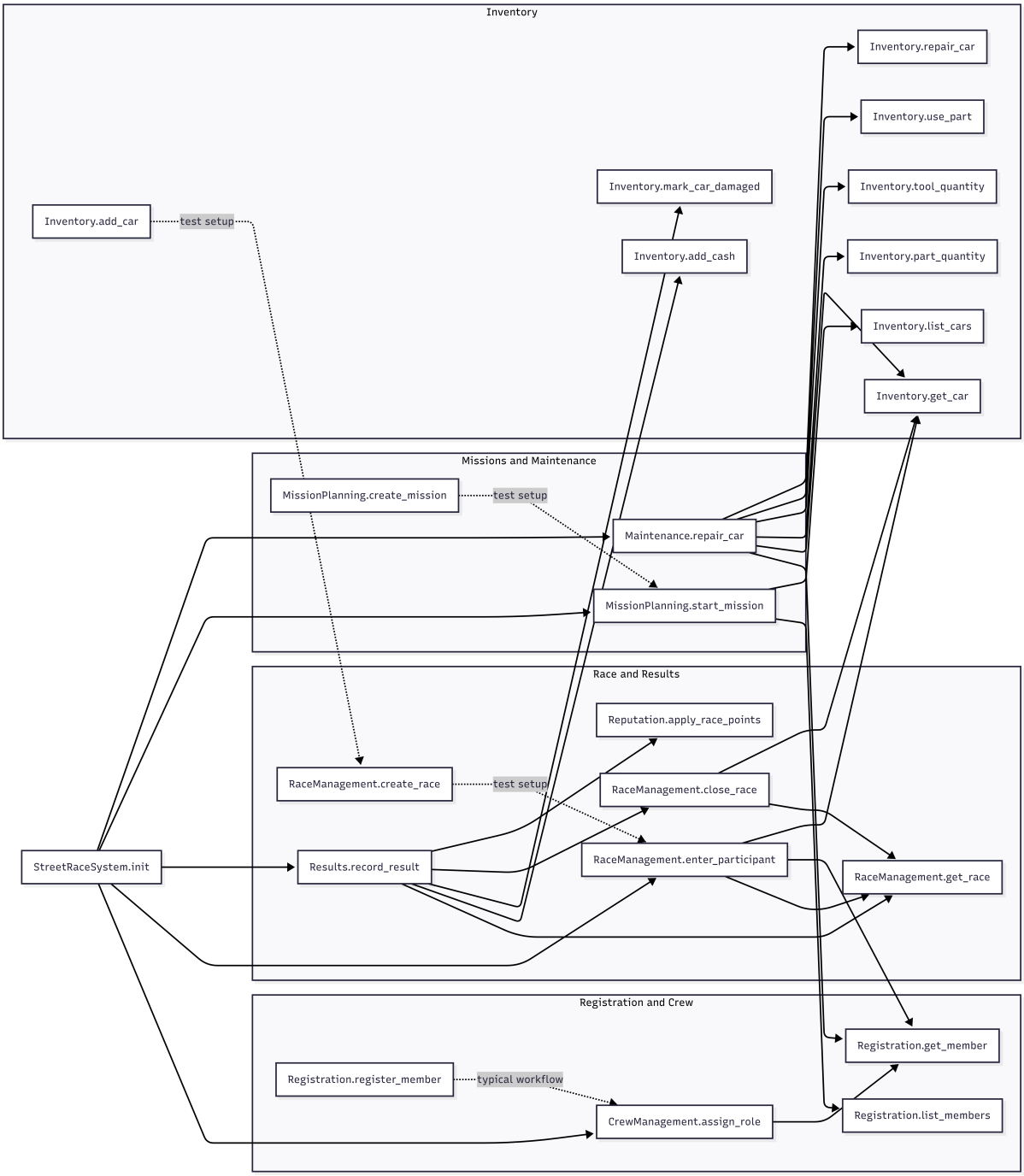

Figure 1: Integration Call Graph

3. **Reject non-driver participant**
   Modules: Registration, Crew Management, Race Management.
   Expected result: registered member with non-driver role cannot enter race.
   Actual result: PASS.
   Errors/issues found: None.
   Why needed: verifies role-based eligibility constraints.

4. **Record race result and cash transfer**
   Modules: Race Management, Results, Inventory.
   Expected result: rankings update and prize money is reflected in inventory cash.
   Actual result: PASS.
   Errors/issues found: None.
   Why needed: confirms results-to-inventory data flow.

5. **Block mission when mechanic missing**
   Modules: Mission Planning, Registration/Crew Management, Inventory.
   Expected result: mission cannot start for damaged car without mechanic availability.
   Actual result: PASS.
   Errors/issues found: None.
   Why needed: validates safety and dependency checks across modules.

6. **Start mission with complete crew**
   Modules: Registration, Crew Management, Inventory, Mission Planning.
   Expected result: valid crew and car allow mission state transition to active.
   Actual result: PASS.
   Errors/issues found: None.
   Why needed: validates successful positive-path mission orchestration.

7. **Repair damaged car with maintenance flow**
   Modules: Maintenance, Registration/Crew Management, Inventory.
   Expected result: car health and availability restored, spare parts consumed.
   Actual result: PASS.
   Errors/issues found: None.
   Why needed: verifies additional module integration with inventory and crew roles.

8. **Update reputation after race outcome**
   Modules: Results, Reputation, Race Management.
   Expected result: higher finishers receive better reputation adjustments.
   Actual result: PASS.
   Errors/issues found: None.
   Why needed: validates second additional module and downstream effect of race results.

9. **Reject role assignment for unknown member**
   Modules: Crew Management, Registration.
   Expected result: role assignment denied when member is not registered.
   Actual result: PASS.
   Errors/issues found: None.
   Why needed: reconfirms integrity checks on direct management operations.

4 Execution Summary

Metric	Value
Command	`python -m pytest integration/tests -q`
Result	**9 passed**
Duration	`0.01s`

5 Observations

- Core cross-module workflows are stable in both negative and positive paths.
- Additional modules (Maintenance and Reputation) integrate correctly with required modules.
- No integration test failures were observed in the latest execution.